Mobile Web Application

React Native & TypeScript Course Notes (Chapters 1-8)

> © ***Eng. Abdallah Mahmoud*** *— React Native & TypeScript Course Notes*

Table of Contents

- Chapter 1 — Introduction to React Native & TypeScript
- Chapter 2 — Git, GitHub & Protecting Sensitive Keys
- Chapter 3 — TypeScript & Components
- Chapter 4 — React Native Components & Styling
- Chapter 5 — State, Events & Hooks
- Chapter 6 — Lists, Forms & Navigation
- Chapter 7 — API & Local Storage
- Chapter 8 — Final Project: Task App

Chapter 1 — Introduction to React Native & TypeScript

> © ***Eng. Abdallah Mahmoud*** *— React Native & TypeScript Course Notes*

1. What is React Native?

React Native is a **framework** made by Facebook (Meta) that lets you build **mobile apps** (for Android and iOS) using **JavaScript/TypeScript** and **React**.

Normally, building a mobile app means writing separate, native code for each platform. With React Native, you write **one codebase** in JavaScript/TypeScript that runs on both Android

and iOS.

> *Key idea: "Learn once, write anywhere."*

Real-world apps built with React Native: Instagram, Discord, Shopify, Walmart.

---

2. Key Concepts: Library, Framework, Page, Template, Component, Web vs App

Before going further, here are a few words you'll see constantly in React Native (and in web dev in general). Short definitions now — you'll see them in action in later chapters.

Library vs Framework

	Library	Framework
Who's in control	**You** call the library's functions when you need them	The **framework** calls your code (you plug into its structure)
Nickname	"A tool you use"	"A skeleton you build inside"
Examples	React, lodash, axios	React Native, Angular, Next.js, Expo

> ***Quirk to remember:*** *React itself is officially a* ***library*** *(it just handles the UI). But* ***React Native*** *is usually called a* ***framework****, because on top of React it also gives you navigation, native modules, build tooling, and project structure — the whole package needed to ship a mobile app.*

Web vs Application

- **Web (website / web app)** — runs inside a **browser**, built from HTML/CSS/JS, opened by typing a **URL**, rendered as a DOM (see Chapter 4 for what that means).
- **Application (mobile app)** — a program **installed** on a phone (built with React Native/Expo, Swift, Kotlin, etc.), runs **natively** with no browser, no URL bar, and no DOM.

Page vs Screen vs Template vs Component

- **Page** — in the **web world**, one destination tied to a URL (e.g. `/about`). Loading a page usually means the browser fetches/renders a new HTML document (or a React Router "view").
- **Screen** — the **mobile equivalent** of a page. Since there's no browser or URL, we say "screen" instead — a full view the user navigates to (e.g. `HomeScreen`, `DetailsScreen`). You'll build these throughout this course.
- **Template** — a reusable **layout skeleton** that arranges components in a fixed structure (e.g. "header + scrollable list + footer button") without being tied to specific content. Real screens then fill that skeleton with actual data. Think of it as the blueprint; the screen is the finished room.
- **Component** — the smallest reusable **building block** of UI (a button, a card, a whole screen is technically a component too). Covered in full in Chapter 3.

Quick-reference table:

Term	Web World	React Native World
Full destination	Page (has a URL)	Screen (no URL, just a name)
Basic container	`<div>`	`<View>`
Moving between destinations	URL / `react-router` `<Link>`	React Navigation, `navigation.navigate()`
Smallest reusable piece	Component	Component (same idea!)

---

3. React Native vs React.js

Feature	React.js	React Native
Used for	Websites (browser)	Mobile apps (Android/iOS)
Renders to	HTML (`<div>` , `<p>`)	Native components (`<View>` , `<Text>`)

Feature	React.js	React Native
Styling	CSS	JavaScript objects (StyleSheet)
Navigation	React Router	React Navigation
Output	Web page	Real mobile app

Simple example:

React.js (Web):

```
<div>
  <p>Hello Web!</p>
</div>
```

React Native (Mobile):

```
<View>
  <Text>Hello Mobile!</Text>
</View>
```

> *Same idea (components + JSX), different building blocks.*

---

4. Advantages of React Native

- **One codebase** for Android + iOS
- **Faster development** than writing separate native apps for each platform
- **Hot Reload** — see changes instantly without restarting the app
- **Large community** and many ready-made libraries
- **Uses JavaScript/TypeScript**, which many developers already know
- Can still access **native features** (camera, GPS, etc.)

---

5. Setting up Expo

Expo is a tool that makes React Native easier for beginners. It handles a lot of the complicated native setup for you.

What is an SDK?

An **SDK** (**S**oftware **D**evelopment **K**it) is a bundle of tools, libraries, and pre-built code for building apps on a specific platform. It's a bigger toolbox than a single library — an SDK usually packages together libraries, command-line tools, and documentation all at once.

The **Expo SDK** specifically is a large collection of ready-made native APIs (camera, location, notifications, sensors, fonts, and more) that Expo provides on top of React Native, so you can use device features without writing any native Swift/Kotlin code yourself.

Each Expo SDK release is tied to a specific, tested combination of React, React Native, and other core library versions, so everything is guaranteed to work together. When you create a new project, it comes pre-configured with whichever SDK version (and matching React/React Native versions) is current at the time.

Quick comparison — Library vs Framework vs SDK:

	Library	Framework	SDK
What it is	Reusable code **you call**	Structure that **calls your code**	A full toolkit — libraries + tools + docs for a platform
Example	React, lodash	React Native, Next.js	Expo SDK, Android SDK, iOS SDK

> ***Where to check your own setup:*** *every project has a* `package.json` *file listing the exact versions of* `react`, `react-native`, `typescript`, *etc. it was created with. That file — not a number memorized from a course — is always the source of truth, since Expo SDK, Node.js, and library versions update frequently.*

Requirements

- Install **Node.js** (from nodejs.org)

- A code editor (VS Code recommended)
- **Expo Go** app on your phone (from Play Store / App Store) — lets you preview your app instantly

Install Expo CLI

You don't need to install Expo CLI globally anymore — you can run it directly using a package manager "runner" command. Pick whichever package manager you have installed; they all do the same thing.

Using npx (comes with Node.js — most common for beginners)

```
npx create-expo-app MyFirstApp
```

Using pnpm

```
pnpm create expo-app MyFirstApp
```

Using yarn

```
yarn create expo-app MyFirstApp
```

Using bun

```
bun create expo-app MyFirstApp
```

> *If you're just starting out, stick with **npx** — it works out of the box with Node.js and is what most tutorials use. The others (pnpm, yarn, bun) are alternative package managers that some developers prefer for speed.*

---

6. Creating and Running a React Native Project

Step 1: Create the project

```
npx create-expo-app MyFirstApp
```

Every new Expo project comes with **TypeScript already set up** by default — you'll see `.tsx` files right away, no extra setup needed.

Step 2: Move into the project folder

```
cd MyFirstApp
```

Step 3: Run the project

```
npx expo start
```

This will open a QR code in your terminal/browser.

Step 4: View your app

- Scan the QR code with the **Expo Go** app on your phone, **or**
- Press `a` to open in Android emulator
- Press `i` to open in iOS simulator (Mac only)

Basic Project Structure

Modern Expo projects use **file-based routing** (via **Expo Router**) by default, so your project starts with an `app/` folder instead of a single `App.tsx` :

```
MyFirstApp/
 ├── app/
 │    ├── _layout.tsx     <- Root layout, wraps every screen (like a shar
 │    └── index.tsx       <- The Home screen (the "/" route)
 ├── assets/               <- Images, fonts, etc.
 ├── package.json          <- Project info, scripts & dependencies
 ├── package-lock.json     <- Exact locked versions of every installed pa
 └── tsconfig.json         <- TypeScript settings
```

What each important file/folder actually does:

- **`package.json`** — the project's manifest. It lists the project's `name` and `version`, the `scripts` you can run (like `npm start`), and two dependency lists: `dependencies` (packages your app needs to run, e.g. `react-native`) and `devDependencies` (tools only needed while developing, e.g. `typescript`). Whenever you `npm install <package>`, an entry gets added here.
- **`package-lock.json`** — auto-generated and auto-updated by npm. While `package.json` might say "any compatible version" of a package, `package-lock.json` records the **exact** version (down to the last digit) that was actually installed for every package and sub-dependency, so that you — and everyone else on the team — get identical installs. **You never edit this file by hand**, but you should commit it to version control (more on that in Chapter 2) so builds stay reproducible.
- **`app/index.tsx`** — inside Expo Router, any file named `index.tsx` is the **default screen** for its folder. `app/index.tsx` is the default screen for `app/`, i.e. your Home screen — the equivalent of what used to be the single root `App.tsx`.
- **`app/_layout.tsx`** — a special file (the underscore marks it as configuration, not a screen itself) that defines a **shared layout/wrapper** around the screens in its folder — for example, wrapping them all in a navigator, a theme provider, or a header that should appear on every screen inside. The root `app/_layout.tsx` wraps your *entire* app.

> ***Expo Router vs. React Navigation:*** *this course teaches* ***React Navigation*** *directly (Chapter 6) because seeing* `Stack.Navigator` / `Tab.Navigator` *explicitly is the clearest way to learn how navigation actually works.* ***Expo Router*** *(the* `app/_layout.tsx + index.tsx` *structure above) is a newer, file-based way of organizing screens — similar in spirit to how Next.js routes pages by folder/file name — and it's built* ***on top of*** *React Navigation under the hood. Once you're comfortable with the concepts in Chapter 6, picking up Expo Router's file-based structure is mostly about where you put files, not new navigation concepts.*

	React Navigation (this course)	Expo Router
How screens are defined	You write `<Stack.Screen name="..." component={...} />` in code	Each file inside `app/` automatically becomes a screen
Shared layout (headers, tabs)	A `Stack.Navigator` / `Tab.Navigator` you render yourself	An `_layout.tsx` file per folder
Good for	Learning exactly how navigation state works	Larger apps, once you already understand the concepts

Your First App.tsx

For simplicity, the rest of this course uses a single root component (`App.tsx`) rather than the `app/` folder structure above — the React/React Native concepts are identical either way, only the file organization differs.

```tsx
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello, React Native with TypeScript!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Quick Recap

- React Native = build mobile apps with JS/TS + React
- Library = you call it; Framework = it calls you. React = library, React Native = framework
- Page (web, has a URL) ≈ Screen (mobile, no URL) — a Template is the layout skeleton; a Component is the smallest reusable piece
- Different from React.js because it renders **native components**, not HTML
- Expo = easiest way to start a React Native project
- SDK = a full toolkit (libraries + tools + docs) for a platform; the Expo SDK bundles native APIs plus matching React/React Native versions — check `package.json` for the exact versions in any given project
- `npx create-expo-app` creates a new project
- `package.json` lists your dependencies; `package-lock.json` locks their exact versions

- Modern projects use file-based routing (`app/_layout.tsx` , `app/index.tsx`) via Expo Router — this course uses React Navigation directly instead, to teach the underlying concepts
- `npx expo start` runs it

Practice Task

1. Install Expo Go on your phone.
2. Create a new project called `PracticeApp` .
3. Change the text inside `App.tsx` to show your name.
4. Run it and see it on your phone.

Chapter 2 — Git, GitHub & Protecting Sensitive Keys

> *© **Eng. Abdallah Mahmoud** — React Native & TypeScript Course Notes*

Before you build bigger projects, learn to **save your code's history** and **share it safely**. This chapter covers version control with Git, hosting code on GitHub, and — most importantly — keeping passwords and API keys out of your public code. Use it on every project from here on, including the Task App in Chapter 8.

---

1. What is Git? What is GitHub?

	Git	GitHub
What it is	A tool on your computer that tracks every change to your code	A website that stores your Git projects online
Works offline?	Yes	No (needs internet)

	Git	GitHub
Analogy	“Save game” checkpoints on your machine	Cloud backup + sharing for those checkpoints

- **Repository (repo)** — a project folder tracked by Git.
- **Commit** — a saved snapshot of your project with a short message.
- **Branch** — a separate line of work (e.g. try a new feature without breaking the main code).
- **Remote** — the online copy of your repo (usually on GitHub).

> *Other hosts like GitLab and Bitbucket work the same way; GitHub is simply the most popular.*

---

2. One-Time Setup

Install Git from git-scm.com, then tell Git who you are:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Create a free account at github.com.

---

3. Basic Git Commands

```
git init                      # start tracking the current folder
git status                    # see what changed
git add .                     # stage ALL changes for the next commit
git commit -m "Add task list" # save a snapshot with a message
```

Send it to GitHub (create an empty repo on github.com first, then copy its URL):

```
git remote add origin https://github.com/your-username/task-app.git
git branch -M main
git push -u origin main       # upload your commits
```

Get a project from GitHub onto your computer:

```
git clone https://github.com/your-username/task-app.git
```

Branches:

```
git branch                    # list branches
git checkout -b feature/search   # create + switch to a new branch
git checkout main             # switch back
git merge feature/search      # bring the branch's work into main
```

Everyday loop: **edit code** → **`git add .`** → **`git commit -m "..."`** → **`git push`**.

Command	Purpose
`git init`	Start a new repo
`git clone <url>`	Copy a repo from GitHub
`git status`	Show changed files
`git add .`	Stage changes
`git commit -m "msg"`	Save a snapshot
`git push`	Upload commits to GitHub
`git pull`	Download the latest commits
`git branch` / `checkout -b`	List / create branches

---

4. `.gitignore` — Files Git Must Not Track

A `.gitignore` file lists files and folders Git should **ignore**. Expo projects already include one. It is important because:

- **`node_modules/`** is huge and can be recreated any time with `npm install` (that is what `package.json` and `package-lock.json` are for — commit those two, not the folder).
- **Build/cache folders** (like `.expo/`) are machine-specific.
- **Secrets** (`.env` files) must never be shared.

```
# dependencies
node_modules/

# Expo
.expo/
dist/
web-build/

# environment variables & secrets
.env
.env.local
.env*.local

# OS files
.DS_Store
```

> *Add `.gitignore` entries **before** your first commit. If a file was already committed, ignoring it later does not remove it from history.*

---

5. Environment Variables & Sensitive Keys

Sensitive keys are things like API keys, database passwords, and tokens. If they land in a public GitHub repo, bots scan for them within minutes and can use them (running up bills,

stealing data).

Rules:

1. **Never** hard-code secrets in your source files.
2. Put them in a `.env` file that is listed in `.gitignore`.
3. Commit a `.env.example` with fake placeholder values so teammates know which variables are needed.

```
# .env  (NEVER committed)
EXPO_PUBLIC_API_URL=https://api.example.com
EXPO_PUBLIC_API_KEY=abc123-my-secret-key
```

```
# .env.example  (safe to commit)
EXPO_PUBLIC_API_URL=
EXPO_PUBLIC_API_KEY=
```

Expo reads variables that start with `EXPO_PUBLIC_` and makes them available in your code:

```
const apiUrl = process.env.EXPO_PUBLIC_API_URL;

const res = await fetch(`${apiUrl}/tasks`); // fetch is covered in Chapte
```

> ***Important warning:*** *anything prefixed* `EXPO_PUBLIC_` *is bundled* ***inside the app****, so users can extract it. Truly secret keys (payment secrets, admin database passwords) should live on a* ***backend server****, never in a mobile app.* `.env` *keeps keys out of GitHub — it does not make them secret inside the app.*

If you accidentally committed a secret: treat it as compromised. **Revoke/regenerate the key** in the provider's dashboard right away — deleting the file afterwards is not enough, because Git keeps history.

Quick Recap

- **Git** tracks changes locally; **GitHub** stores repos online
- Loop: `git add .` → `git commit -m "..."` → `git push`
- `git clone` downloads a repo; branches let you work safely on new features
- `.gitignore` keeps `node_modules/`, caches, and `.env` out of Git
- Commit `package.json` and `package-lock.json`; never commit `.env`
- Use `.env` + `.env.example`; if a key leaks, **revoke it**

Practice Task

1. Run `git init` in your `PracticeApp` folder from Chapter 1 and make your first commit.
2. Confirm `.env` and `node_modules/` are in `.gitignore`.
3. Create a GitHub repo and push your project.
4. Create a `feature/change-greeting` branch, change the text in `App.tsx`, and merge it back into `main`.

Chapter 3 — TypeScript & Components

> © ***Eng. Abdallah Mahmoud*** *— React Native & TypeScript Course Notes*

1. TypeScript Data Types

TypeScript adds **types** to JavaScript, which helps catch mistakes before running the code.

```ts
let username: string = "Ahmed";
let age: number = 20;
let isStudent: boolean = true;
let hobbies: string[] = ["reading", "coding"];
let score: number[] = [90, 85, 70];

// any -> avoid using this when possible
let randomValue: any = "could be anything";

// union types -> value can be more than one type
let id: string | number = 101;
```

---

2. Interfaces and Types

Both `interface` and `type` describe the **shape** of an object.

Interface

```ts
interface User {
  name: string;
  age: number;
  isStudent: boolean;
}

const student: User = {
  name: "Sara",
  age: 22,
  isStudent: true,
};
```

Type

```
type Product = {
  title: string;
  price: number;
};

const item: Product = {
  title: "Laptop",
  price: 999,
};
```

> *For beginners: use `interface` for objects/props, and `type` when combining types.*

Optional Properties

```
interface User {
  name: string;
  email?: string; // optional, can be missing
}
```

---

3. Functions with TypeScript

Regular function

```
// Function with typed parameters and return type
function add(a: number, b: number): number {
  return a + b;
}
```

Arrow Functions (a.k.a. “Fat Arrow” or “Lambda” Functions)

An **arrow function** is just a shorter way to write a function, using the `=>` symbol (which looks like a fat arrow, hence the nickname). You may also hear it called a **lambda function** — that’s just a term from other programming languages for the same idea: a small, unnamed (or quickly assigned) function.

```
// Regular function
function greet(name: string): string {
  return `Hello, ${name}!`;
}

// The exact same thing, written as an arrow function
const greet = (name: string): string => {
  return `Hello, ${name}!`;
};

// If the function body is just one line that returns a value,
// you can drop the { } and the "return" keyword entirely:
const greetShort = (name: string): string => `Hello, ${name}!`;
```

Why arrow functions matter in React Native:

- They are the standard way to write **functional components** (see below).
- They are used constantly for event handlers: `onPress={() => doSomething()}`
- They keep code short and easy to read once you’re used to them.

Style	Example
Regular function	`function add(a: number, b: number) { return a + b; }`
Arrow function	`const add = (a: number, b: number) => { return a + b; }`
Short arrow function (implicit return)	`const add = (a: number, b: number) => a + b;`

Function with optional/default parameter

```
function multiply(a: number, b: number = 2): number {
  return a * b;
}
```

---

4. Functional Components

A **component** is a reusable, self-contained piece of UI — think of it like a custom building block (a button, a card, a whole screen) that you can reuse anywhere in your app.

A **functional component** is simply a **function that returns JSX** (the UI to display). In modern React and React Native, almost all components are written this way (older code used "class components", which you don't need to learn as a beginner).

```
function Welcome() {
  return (
    <View>
      <Text>Welcome to the app!</Text>
    </View>
  );
}

export default Welcome;
```

Since arrow functions are so common in React Native, functional components are very often written like this instead:

```
const Welcome = () => {
  return (
    <View>
      <Text>Welcome to the app!</Text>
    </View>
  );
};

export default Welcome;
```

> *Both versions above do **exactly the same thing**. `Welcome` is a function; it just happens to return JSX instead of a number or string.*

Why use components at all?

- **Reusability** — write a `Button` once, use it 20 times.
- **Organization** — break a big screen into small, manageable pieces.
- **Readability** — `<ProfileCard />` is easier to understand than 40 lines of raw JSX repeated everywhere.

---

5. Props

Props (short for "properties") let you pass data **into** a component from the outside — exactly like passing arguments into a function.

```
function Greeting(props: { name: string }) {
  return <Text>Hello, {props.name}!</Text>;
}

// Using the component
<Greeting name="Ali" />
<Greeting name="Amina" />
```

Here, the **same** `Greeting` component is reused, but each time it displays something different because it received different **props**.

Why are Props Important?

- They make components **reusable**. Without props, `Greeting` could only ever say one hardcoded name — with props, it can greet *anyone*.
- They let data flow from a **parent component** down to a **child component**, which is how information moves through a React Native app.
- They keep components **generic and flexible** — one `Button` component can have different `title`, `color`, or `onPress` behavior every time it's used.

Think of props like the settings you hand to a component when you use it — similar to how you might pass arguments into a function to control what it does.

Why are Props Read-Only?

A component is **not allowed to change (mutate) its own props**. Props flow in **one direction only**: from parent → child. This is often called **"one-way data flow"**.

```
function Greeting(props: { name: string }) {
  // This is NOT allowed and will cause an error/bug:
  // props.name = "Someone else";

  return <Text>Hello, {props.name}!</Text>;
}
```

Why does this rule exist?

- **Predictability** — if a child component could silently change the data it receives, the parent would never know why its own data changed. Bugs would be very hard to track down.
- **Data flows in one clear direction** — parent → child. This makes large apps much easier to reason about, because you always know *where* data comes from.
- **If a component needs to change its own data over time** (e.g. a counter that increases), it should **not** use props for that — it should use **state** instead (covered in the next

chapter). Props are for data passed in from outside; **state** is for data a component manages and updates itself.

> *Simple rule to remember:* ***Props = read-only input from a parent. State = data the component owns and can change itself.***

---

6. Typing Component Props

The recommended way is to define an **interface** for props:

```
interface GreetingProps {
  name: string;
  age?: number; // optional
}

const Greeting = ({ name, age }: GreetingProps) => {
  return (
    <View>
      <Text>Hello, {name}!</Text>
      {age && <Text>You are {age} years old.</Text>}
    </View>
  );
};

export default Greeting;
```

Usage:

```
<Greeting name="Fatima" age={19} />
```

---

7. JSX vs TSX

JSX	TSX
Used in plain JavaScript React files	Used in TypeScript React files
File extension: `.jsx`	File extension: `.tsx`
No type checking	Has type checking for props, state, etc.

JSX example (.jsx):

```
function Hello() {
  return <Text>Hi there</Text>;
}
```

TSX example (.tsx):

```
function Hello(): JSX.Element {
  return <Text>Hi there</Text>;
}
```

> *In this course, we always use **.tsx** files because we're using TypeScript.*

Quick Recap

- TypeScript adds types: `string`, `number`, `boolean`, arrays, unions
- `interface` / `type` describe object shapes (great for props)
- Functional components are plain functions that return JSX
- Props pass data into components — always type them
- `.tsx` = TypeScript + JSX

Practice Task

1. Create an interface `Book` with `title: string` and `pages: number`.
2. Create a component `BookCard` that accepts `Book` as props and displays the info.
3. Use it in `App.tsx` with your favorite book.

Chapter 4 — React Native Components & Styling

> © ***Eng. Abdallah Mahmoud*** *— React Native & TypeScript Course Notes*

0. What is the DOM? (And Why React Native Doesn't Have One)

DOM = **Document Object Model**. When a browser loads an HTML page, it builds a **tree** of objects representing every element on the page (`<div>` , `<p>` , `<img>` , etc.). JavaScript can then read and change that tree — that's what `document.getElementById(...)` is reaching into.

Example — this HTML:

```
<html>
  <body>
    <div>
      <h1>My Page</h1>
      <p>Hello world</p>
    </div>
  </body>
</html>
```

...becomes this DOM tree:

```
html
 └── body
      └── div
           ├── h1 ("My Page")
           └── p ("Hello world")
```

Every element is a “node” with a parent and (optionally) children — that’s why it’s called a tree.

Very briefly, how a browser turns this into what you see on screen (just for context, not a deep dive — this is browser-internals territory beyond a beginner course): the browser **parses** the HTML into this DOM tree, combines it with CSS rules, then **paints** the result to pixels on the screen. Whenever JavaScript changes the DOM, the browser repeats part of that process to update what’s visible.

How React.js (web) uses it:

- Your JSX compiles down to instructions that create/update real HTML elements in the DOM.
- React keeps an in-memory copy called the **Virtual DOM**. When state changes, React compares (“diffs”) the new Virtual DOM against the previous one, then updates only the parts of the *real* DOM that actually changed — this is why React is fast.

How React Native is different:

- There is **no browser and no DOM at all** in a React Native app — nothing is rendered as HTML.
- Your JSX (`<View>` , `<Text>` , etc.) is instead translated into real **native UI elements** (an actual Android `View` / iOS `UIView`) through React Native’s rendering layer (the “bridge”, or the newer JSI-based architecture).
- The React concepts you already know — components, props, state, re-rendering — work exactly the same. Only the *final output* differs: native widgets instead of DOM nodes.

Quick comparison:

	React.js (Web)	React Native
Final output	HTML elements in the browser DOM	Real native UI elements (Android/iOS)
Diffing mechanism	Virtual DOM → real DOM	Virtual DOM → native UI tree
Can you use `document.getElementById`?	Yes	No — there's no document/DOM to query

---

1. View

`View` is like a `<div>` in web development — a container for other components.

```
import { View, Text } from 'react-native';

function Box() {
  return (
    <View style={{ padding: 20, backgroundColor: 'lightblue' }}>
      <Text>I am inside a View</Text>
    </View>
  );
}
```

---

2. SafeAreaView

Phones have notches, camera cutouts, rounded corners, and system bars (the status bar on top, the home indicator on the bottom). A plain `View` doesn't know about any of that — its content can end up hidden behind the notch or the status bar.

`SafeAreaView` is a container, just like `View`, but it automatically adds padding so its content stays inside the visible, "safe" area of the screen.

```
import { SafeAreaView, Text, StyleSheet } from 'react-native';

function Screen() {
  return (
    <SafeAreaView style={styles.container}>
      <Text>This text won't be hidden behind the notch or status bar</Tex
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1 },
});
```

> ***Which import to use?*** *React Native ships a basic* `SafeAreaView` *, but the more reliable, actively-maintained version comes from the* `react-native-safe-area-context` *package (already installed as part of React Navigation's setup — see Chapter 6):*
>
> ```
> import { SafeAreaView } from 'react-native-safe-area-context';
> ```
>
> *It works the same way, just handles more edge cases (like rotating the device) correctly.*

Rule of thumb: wrap your top-level screen content in a `SafeAreaView` instead of a plain `View` whenever that content touches the top or bottom edge of the screen (like a header or a bottom input bar) — you'll see this used in the Chapter 8 final project.

---

3. Text

`Text` is used to display any text. All text **must** be wrapped in a `<Text>` component (unlike HTML where text can float freely).

```
<Text style={{ fontSize: 18, color: 'blue' }}>Hello World</Text>
```

---

4. Image

`Image` is used to show pictures — from local files or the internet.

```
import { Image } from 'react-native';

// Local image
<Image source={require('./assets/logo.png')} style={{ width: 100, height:

// Image from internet
<Image
  source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
  style={{ width: 100, height: 100 }}
/>
```

---

5. Button

`Button` is a simple, ready-made clickable button.

```
import { Button, Alert } from 'react-native';

<Button
  title="Click Me"
  onPress={() => Alert.alert('Button pressed!')}
/>
```

> *Note: `Button` has limited styling. For custom-styled buttons, use `TouchableOpacity` + `Text`.*

```
import { TouchableOpacity, Text } from 'react-native';

<TouchableOpacity style={{ backgroundColor: 'green', padding: 10 }}>
  <Text style={{ color: 'white' }}>Custom Button</Text>
</TouchableOpacity>
```

---

6. TextInput

`TextInput` lets users type text (like an `<input>` in HTML).

```
import { TextInput } from 'react-native';
import { useState } from 'react';

function NameInput() {
  const [text, setText] = useState('');

  return (
    <TextInput
      placeholder="Enter your name"
      value={text}
      onChangeText={setText}
      style={{ borderWidth: 1, padding: 10 }}
    />
  );
}
```

---

7. ScrollView

`ScrollView` lets content scroll when it's too big for the screen.

```
import { ScrollView, Text } from 'react-native';

<ScrollView>
  <Text>Line 1</Text>
  <Text>Line 2</Text>
  {/* ... imagine 50 more lines ... */}
</ScrollView>
```

> *For long lists, use `FlatList` instead (covered in Chapter 6) — it's more efficient.*

---

8. StyleSheet

`StyleSheet.create()` is the recommended way to write styles (similar to CSS, but as a JS object).

```
import { StyleSheet, View, Text } from 'react-native';

function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#333',
  },
});
```

Why use StyleSheet instead of inline styles?

- Cleaner code
- Slightly better performance
- Easier to reuse styles

---

9. Flexbox

Flexbox is the layout system React Native uses to arrange components on the screen — how they're positioned, sized, and spaced. If you've never seen it before, think of it as a set of rules that answer: *"Should my components sit in a row or a column? Should they be centered, spread out, or squeezed together?"*

The Main Axis and the Cross Axis

Every `View` with Flexbox has two axes:

- The **main axis** — the primary direction items are laid out (controlled by `flexDirection`)
- The **cross axis** — the direction perpendicular to the main axis

> *Important default: in React Native (unlike CSS on the web), `flexDirection` defaults to **`column`** — meaning items stack **top to bottom** unless you say otherwise.*

`flexDirection`

Controls the direction of the main axis.

```
flexDirection: 'column' // default — items stacked vertically (top to bot
flexDirection: 'row'    // items placed side by side (left to right)
```

`justifyContent` — spacing along the main axis

```
const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    justifyContent: 'space-between', // pushes items to the edges, even g
  },
});
```

Common values: `flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`

`alignItems` — alignment along the cross axis

```
const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    alignItems: 'center', // vertically centers items when direction is '
  },
});
```

Common values: `flex-start`, `center`, `flex-end`, `stretch`

`flex: 1` — sharing available space

`flex: 1` tells a component to **grow and fill the remaining space** in its container. It's one of the most-used properties in React Native.

```
<View style={{ flex: 1, backgroundColor: 'yellow' }} />
```

If a parent `View` has `flex: 1`, it takes up the entire screen (since the root container has no size limits above it).

Quick reference table:

Property	What it controls
`flexDirection`	Direction of layout: `row` or `column` (default)
`justifyContent`	Spacing along the **main axis**
`alignItems`	Alignment along the **cross axis**
`flex: 1`	Makes a component grow to fill available space

Example — Two boxes side by side, centered, evenly spaced

```
<View style={{ flex: 1, flexDirection: 'row', justifyContent: 'space-arou
  <View style={{ width: 50, height: 50, backgroundColor: 'red' }} />
  <View style={{ width: 50, height: 50, backgroundColor: 'blue' }} />
</View>
```

> *Beginner tip: the fastest way to learn Flexbox is to experiment — change `flexDirection` from `column` to `row` and watch how the layout flips, then try different `justifyContent` / `alignItems` values.*

---

10. Colors, Fonts, Margins, and Padding

```
const styles = StyleSheet.create({
  box: {
    backgroundColor: '#4CAF50',  // background color
    padding: 16,                 // space inside the box
    margin: 10,                  // space outside the box
  },
  text: {
    fontSize: 20,
    fontWeight: 'bold',
    fontFamily: 'System', // custom fonts can be loaded too
    color: '#ffffff',
  },
});
```

Margin vs Padding (quick memory trick):

- **Padding** = space **inside** the box (between border and content)
- **Margin** = space **outside** the box (between box and other elements)

Quick Recap

- The DOM is a web/browser concept (tree of HTML elements); React Native has **no DOM** — it renders real native UI elements instead
- `View` = container, `Text` = text, `Image` = pictures
- `SafeAreaView` = a container that keeps content clear of notches, status bars, and home indicators
- `Button` / `TouchableOpacity` = clickable actions
- `TextInput` = user text input
- `ScrollView` = scrollable content
- `StyleSheet.create()` = organize your styles
- Flexbox = layout system (row/column, justify/align)
- Padding = inside space, Margin = outside space

Practice Task

Build a simple **Profile Card** with:

- An `Image` (profile picture)
- A `Text` (name)
- A `Text` (bio)
- A `TouchableOpacity` button that says "Follow"

Style it using `StyleSheet` and Flexbox.

Chapter 5 — State, Events & Hooks

> *© **Eng. Abdallah Mahmoud** — React Native & TypeScript Course Notes*

0. What is a Hook?

A **Hook** is a special function in React that lets a **functional component** do things that plain functions normally can't — like remembering data between renders, running code at

specific times, or reaching into a component directly.

You can always recognize a hook because **its name starts with `use`**: `useState`, `useEffect`, `useRef`, `useMemo`, and so on.

Why do hooks exist?

- Functional components are just regular functions — normally, a function forgets everything the moment it finishes running.
- Hooks give functional components a way to **"hook into"** React's features (memory, lifecycle events, references) without needing to write more complex class-based components.

Two simple rules for using hooks:

1. Only call hooks at the **top level** of a component (not inside loops, conditions, or nested functions).
2. Only call hooks from **React functional components** (or from other custom hooks).

Now let's look at the most common hooks one by one.

1. useState

`useState` is the most common hook. It lets a component **remember and update data** (called "state") across renders — something a normal function couldn't do on its own. When state changes, React automatically re-renders the component to reflect the new value on screen.

```tsx
import { useState } from 'react';
import { View, Text, Button } from 'react-native';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Increase" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

How it works:

- `count` → the current value
- `setCount` → function to update the value
- `useState(0)` → the starting value is `0`

---

2. Typing State with TypeScript

```tsx
const [count, setCount] = useState<number>(0);
const [name, setName] = useState<string>('');
const [isVisible, setIsVisible] = useState<boolean>(false);

// For objects, define an interface
interface User {
  name: string;
  age: number;
}

const [user, setUser] = useState<User | null>(null);
```

> *If you don't give a type, TypeScript usually **guesses** it from the initial value. But for things like `null` or arrays, it's better to be explicit.*

---

3. Handling Button Events

```
import { Button, Alert } from 'react-native';

function App() {
  const handlePress = () => {
    Alert.alert('You tapped the button!');
  };

  return <Button title="Tap Me" onPress={handlePress} />;
}
```

You can also pass data to the handler:

```
const handlePress = (message: string) => {
  Alert.alert(message);
};

<Button title="Say Hi" onPress={() => handlePress('Hi there!')} />
```

---

4. Text Input Events

```
import { useState } from 'react';
import { TextInput, Text, View } from 'react-native';

function NameForm() {
  const [name, setName] = useState<string>('');

  return (
    <View>
      <TextInput
        placeholder="Type your name"
        value={name}
        onChangeText={(text) => setName(text)}
        style={{ borderWidth: 1, padding: 8 }}
      />
      <Text>Hello, {name}</Text>
    </View>
  );
}
```

- `onChangeText` fires every time the user types a letter.
- `value={name}` keeps the input in sync with state (this is called a "controlled input").

---

5. Controlled vs Uncontrolled Components

- **Controlled component** — the component's value is **driven by React state**. The `TextInput` above is controlled: its `value` always comes from `name`, and every keystroke updates `name` via `onChangeText`. React state is the "single source of truth."
- **Uncontrolled component** — the component **manages its own value internally**, and React doesn't track every keystroke. You only read the value when you need it (e.g. using a `ref`).

```tsx
// Uncontrolled example — no `value` prop, no state on every keystroke
import { useRef } from 'react';
import { TextInput, Button } from 'react-native';

function UncontrolledForm() {
  const inputRef = useRef<TextInput>(null);

  const handleSubmit = () => {
    // Reading the current value only when needed
    console.log(inputRef.current);
  };

  return (
    <>
      <TextInput ref={inputRef} placeholder="Type here" />
      <Button title="Submit" onPress={handleSubmit} />
    </>
  );
}
```

Quick comparison:

	Controlled	Uncontrolled
Value lives in	React state (`useState`)	The component itself (read via `ref` when needed)
Re-renders on every keystroke?	Yes	No
Easiest for	Validation, formatting-as-you-type, disabling a button until valid	Simple forms where you only need the value on submit

> ***Rule of thumb for this course:*** *default to* ***controlled*** *components (as used throughout this book) — they're more predictable and are what you'll see in almost all React Native code. Uncontrolled is a useful escape hatch for performance-sensitive forms with many fields.*

---

6. useEffect

`useEffect` runs code **after the component renders** — useful for things like fetching data, timers, or logging.

```
import { useEffect, useState } from 'react';
import { Text } from 'react-native';

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const interval = setInterval(() => {
      setTime(new Date());
    }, 1000);

    return () => clearInterval(interval); // cleanup when component unmou
  }, []); // empty array = run only once when component loads

  return <Text>{time.toLocaleTimeString()}</Text>;
}
```

The dependency array `[]` controls when useEffect runs:

Dependency array	When it runs
`[]`	Once, when component first loads
`[count]`	Every time `count` changes

Dependency array	When it runs
(none)	After every render (rarely used)

---

7. useMemo

`useMemo` is a hook that **remembers (caches) the result of an expensive calculation**, so it doesn't have to be re-calculated on every single render — only when its dependencies actually change.

```
import { useMemo, useState } from 'react';
import { View, Text, Button } from 'react-native';

function ExpensiveList() {
  const [count, setCount] = useState(0);
  const [numbers] = useState([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);

  // Imagine this calculation is very slow/expensive
  const total = useMemo(() => {
    console.log('Calculating total...');
    return numbers.reduce((sum, n) => sum + n, 0);
  }, [numbers]); // only recalculates if "numbers" changes

  return (
    <View>
      <Text>Total: {total}</Text>
      <Text>Count: {count}</Text>
      <Button title="Increase Count" onPress={() => setCount(count + 1)} /
    </View>
  );
}
```

Notice that pressing the button changes `count` and re-renders the component, but **"Calculating total..." is not printed again**, because `numbers` never changed — `useMemo`

skipped recalculating and reused the cached result.

When to use it:

- When you have a genuinely slow calculation (sorting/filtering a big list, heavy math).
- As a beginner, you won't need `useMemo` often — but it's good to recognize what it does when you see it.

> *Simple rule: `useState` remembers a **value**. `useEffect` runs **side-effect code**. `useMemo` remembers the **result of a calculation**.*

8. Conditional Rendering

Show different content based on a condition.

```
function Greeting({ isLoggedIn }: { isLoggedIn: boolean }) {
  return (
    <View>
      {isLoggedIn ? (
        <Text>Welcome back!</Text>
      ) : (
        <Text>Please log in.</Text>
      )}
    </View>
  );
}
```

Using `&&` for simple show/hide:

```
{isLoading && <Text>Loading...</Text>}
```

---

9. useRef Basics

`useRef` is a hook that gives you a "box" to hold a value that:

- **Persists** across renders (like state), **but**
- **Does NOT cause a re-render** when it changes (unlike state).

It has two main uses:

1. **Referencing an actual component** on screen — e.g. to focus a `TextInput` or scroll a list.
2. **Storing a value that changes over time but shouldn't trigger a re-render** — e.g. a timer ID, a previous value, a counter used only internally.

```
import { useRef } from 'react';
import { TextInput, Button, View } from 'react-native';

function FocusInput() {
  const inputRef = useRef<TextInput>(null);

  return (
    <View>
      <TextInput ref={inputRef} placeholder="Type here" style={{ borderWi
      <Button title="Focus Input" onPress={() => inputRef.current?.focus(
    </View>
  );
}
```

Common uses for `useRef` :

- Focus an input
- Store a value across renders without re-rendering (like a timer ID)

Quick Recap

- A **Hook** = a special function (starts with `use`) that lets functional components remember data or tap into React features
- `useState` = store & update data that affects the UI (triggers a re-render)
- Always type your state (`useState<number>(0)`)

- Events like `onPress` and `onChangeText` handle user interaction
- **Controlled** component = value driven by state; **uncontrolled** = component manages its own value, read via `ref`
- `useEffect` = run code after render (fetching, timers)
- `useMemo` = remembers the result of an expensive calculation, avoids recalculating unnecessarily
- Conditional rendering = show/hide UI based on a condition
- `useRef` = reference a component or store a value **without** causing a re-render

Practice Task

Build a simple **Like Button**:

- Use `useState` to track `liked: boolean` and `likeCount: number`
- When pressed, toggle `liked` and increase/decrease `likeCount`
- Show "Liked" or "Like" conditionally based on `liked`

Chapter 6 — Lists, Forms & Navigation

> *© **Eng. Abdallah Mahmoud** — React Native & TypeScript Course Notes*

1. FlatList

`FlatList` is used to efficiently display **long lists** of data (better than `ScrollView` for big lists, because it only renders items on screen).

```
import { FlatList, Text, View } from 'react-native';

const fruits = ['Apple', 'Banana', 'Mango', 'Orange'];

function FruitList() {
  return (
    <FlatList
      data={fruits}
      keyExtractor={(item, index) => index.toString()}
      renderItem={({ item }) => <Text>{item}</Text>}
    />
  );
}
```

Key props:

Prop	Purpose
`data`	The array of items
`renderItem`	How each item looks
`keyExtractor`	Unique key for each item (like React's `key`)

---

2. Rendering Arrays

You can also render arrays with plain JavaScript `.map()` (fine for small lists):

```
{fruits.map((fruit, index) => (
  <Text key={index}>{fruit}</Text>
))}
```

> *Use `.map()` for small, simple lists. Use `FlatList` for larger or scrollable lists.*

---

3. Typing List Data

```
interface Task {
  id: string;
  title: string;
  done: boolean;
}

const tasks: Task[] = [
  { id: '1', title: 'Learn React Native', done: false },
  { id: '2', title: 'Build a project', done: false },
];

function TaskList() {
  return (
    <FlatList
      data={tasks}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => <Text>{item.title}</Text>}
    />
  );
}
```

---

4. Creating Forms

A simple form combines multiple `TextInput`s with state.

```
import { useState } from 'react';
import { View, TextInput, Button, Text } from 'react-native';

function SignupForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');

  const handleSubmit = () => {
    console.log('Submitted:', name, email);
  };

  return (
    <View>
      <TextInput placeholder="Name" value={name} onChangeText={setName} s
      <TextInput placeholder="Email" value={email} onChangeText={setEmail
      <Button title="Submit" onPress={handleSubmit} />
    </View>
  );
}
```

---

5. Form Validation

Basic validation checks input before submitting.

```
const [email, setEmail] = useState('');
const [error, setError] = useState('');

const handleSubmit = () => {
  if (!email.includes('@')) {
    setError('Please enter a valid email');
    return;
  }
  setError('');
  console.log('Form is valid!');
};

return (
  <View>
    <TextInput placeholder="Email" value={email} onChangeText={setEmail} /
    {error ? <Text style={{ color: 'red' }}>{error}</Text> : null}
    <Button title="Submit" onPress={handleSubmit} />
  </View>
);
```

Common checks:

- Field not empty: `if (name.trim() === '')`
- Valid email: `if (!email.includes('@'))`
- Minimum length: `if (password.length < 6)`

---

6. What is Routing / Navigation?

Routing is the general idea of deciding which screen/page to show, and letting the user move between them.

On the web, routing is URL-based: each page has an address (`/home` , `/profile/5`), and clicking a `<Link>` (e.g. with `react-router-dom`) changes the URL and swaps the visible page. This is often just called **"linking"**.

In React Native, there's no URL bar for the user — routing is handled entirely by a navigation library in JS. Instead of `<Link href="/profile">`, you write `navigation.navigate('Profile')`. (React Navigation *can* still respond to real URLs for things like opening the app from a website or a notification — this is called **deep linking**, an optional advanced topic.)

The three common navigator patterns:

Navigator	What it looks like	Best for
Stack	Screens stacked like cards; new screen slides in, back button pops it off	Drill-down flows: List → Details → Edit
Tabs	A persistent bar (usually at the bottom) letting you jump directly between a few sections	A handful of top-level sections: Home, Search, Profile
Drawer	A hidden side menu that slides in from the edge	Apps with many top-level sections

> *These aren't mutually exclusive — real apps often **nest** them, e.g. Tabs at the root, with each tab containing its own Stack for drilling into details.*

This course focuses on **Stack** (below) and **Tabs** (Section 9), since together they cover most beginner apps.

---

7. React Navigation

`React Navigation` is the standard library for moving between screens in a React Native app.

Install

```
npm install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context
```

8. Stack Navigation

“Stack” navigation is like a stack of cards — you push a new screen on top, and can go back.

```tsx
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stac
import { View, Text, Button } from 'react-native';

const Stack = createNativeStackNavigator();

function HomeScreen({ navigation }: any) {
  return (
    <View>
      <Text>Home Screen</Text>
      <Button title="Go to Details" onPress={() => navigation.navigate('D
    </View>
  );
}

function DetailsScreen() {
  return (
    <View>
      <Text>Details Screen</Text>
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

9. Tab Navigation

While **Stack** navigation pushes screens on top of each other, **Tab** navigation shows a persistent bar (usually at the bottom) so the user can jump straight to a section at any time — each tab keeps its own state in the background.

Install

```
npm install @react-navigation/bottom-tabs
```

```
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { View, Text } from 'react-native';

const Tab = createBottomTabNavigator();

function HomeScreen() {
  return (
    <View>
      <Text>Home Screen</Text>
    </View>
  );
}

function ProfileScreen() {
  return (
    <View>
      <Text>Profile Screen</Text>
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen name="Home" component={HomeScreen} />
        <Tab.Screen name="Profile" component={ProfileScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

> ***Combining Stack + Tabs:*** *a common real-app pattern is a* `Tab.Navigator` *at the root, where one of the tabs (e.g. "Home") is itself a* `Stack.Navigator` *— so tapping a list item inside the Home tab can still push a Details screen, while the tab bar stays visible.*

---

10. Screen Options (Customizing Headers & Tabs)

Every navigator lets you customize how its screens **look**, using the `options` prop on a `Screen`, or `screenOptions` on the `Navigator` itself to apply the same settings to *every* screen at once.

Customizing a Stack header

```
<Stack.Screen
  name="Details"
  component={DetailsScreen}
  options={{
    title: 'Task Details',        // header text
    headerStyle: { backgroundColor: '#4F46E5' },
    headerTintColor: '#fff',      // back-arrow + title color
    headerShown: true,            // set false to hide the header entirel
  }}
/>
```

Apply the same style to **every** screen in the navigator at once with `screenOptions` on the `Navigator`:

```
<Stack.Navigator
  screenOptions={{
    headerStyle: { backgroundColor: '#4F46E5' },
    headerTintColor: '#fff',
  }}
>
  <Stack.Screen name="Home" component={HomeScreen} />
  <Stack.Screen name="Details" component={DetailsScreen} />
</Stack.Navigator>
```

Customizing Tab icons and labels

```
import { Ionicons } from '@expo/vector-icons';

<Tab.Navigator
  screenOptions={({ route }) => ({
    tabBarIcon: ({ color, size }) => {
      const iconName = route.name === 'Home' ? 'home' : 'person';
      return <Ionicons name={iconName} size={size} color={color} />;
    },
    tabBarActiveTintColor: '#4F46E5',
    tabBarInactiveTintColor: 'gray',
  })}
>
  <Tab.Screen name="Home" component={HomeScreen} />
  <Tab.Screen name="Profile" component={ProfileScreen} />
</Tab.Navigator>
```

Common options you'll reach for:

Option	What it does
`title`	Text shown in the header (or tab label if no `tabBarLabel` set)
`headerShown`	Show/hide the header for a screen

Option	What it does
`headerStyle` / `headerTintColor`	Header background color / text+icon color
`tabBarIcon`	Function returning the icon shown in a tab
`tabBarActiveTintColor` / `tabBarInactiveTintColor`	Colors for the selected vs unselected tab

> ***Screen vs Navigator options:*** *put an option on a single* `Screen` *when only that screen should look different; put it on* `screenOptions` *(on the* `Navigator` *) when it should apply to all screens inside it.*

---

11. Passing Parameters

You can pass data from one screen to another.

```
// From Home Screen
navigation.navigate('Details', { userId: 42, userName: 'Amina' });

// In Details Screen
function DetailsScreen({ route }: any) {
  const { userId, userName } = route.params;
  return (
    <View>
      <Text>User ID: {userId}</Text>
      <Text>User Name: {userName}</Text>
    </View>
  );
}
```

---

12. Type-Safe Navigation

Instead of using `any`, define types for your navigation stack.

```
type RootStackParamList = {
  Home: undefined;
  Details: { userId: number; userName: string };
};

import { NativeStackScreenProps } from '@react-navigation/native-stack';

type DetailsProps = NativeStackScreenProps<RootStackParamList, 'Details'>

function DetailsScreen({ route, navigation }: DetailsProps) {
  const { userId, userName } = route.params;
  return (
    <View>
      <Text>{userName} (ID: {userId})</Text>
    </View>
  );
}
```

This gives you **autocomplete and error-checking** when navigating and passing params.

Quick Recap

- `FlatList` = efficient list rendering with `data`, `renderItem`, `keyExtractor`
- Type your list data with interfaces
- Forms = `TextInput` + `useState`, validate before submit
- Routing = deciding which screen to show; on the web it's URL-based ("linking"), in React Native it's handled by React Navigation via `navigation.navigate(...)`
- React Navigation = the standard library for moving between screens
- **Stack** Navigator = push/pop screens like a stack of cards (drill-down flows)
- **Tab** Navigator = persistent bar to jump directly between top-level sections

- `options` (per-screen) and `screenOptions` (whole navigator) customize headers, colors, and tab icons
- Pass data between screens with `navigation.navigate('Screen', {params})`
- Type-safe navigation avoids bugs with `RootStackParamList`

Practice Task

Create a two-screen app:

1. **Home Screen**: a `FlatList` of student names.
2. Tapping a name navigates to a **Profile Screen** showing that student's name (passed as a parameter).
3. **Bonus:** wrap it in a Tab Navigator with a second tab (e.g. "Settings") alongside your Stack.

Chapter 7 — API & Local Storage

> © ***Eng. Abdallah Mahmoud*** *— React Native & TypeScript Course Notes*

1. Understanding APIs

An **API** (Application Programming Interface) lets your app talk to a server to **get** or **send** data — for example, getting a list of movies or user profiles from the internet.

Think of it like a waiter in a restaurant:

- You (the app) ask the waiter (API) for food (data)
- The waiter goes to the kitchen (server) and brings it back

Concretely, most APIs you'll use are **Web APIs**: you send a request to a **URL** (e.g. `https://api.example.com/users`), and the server sends back **data** — usually as **JSON** (see below).

---

2. What is JSON? (and XML)

JSON (**J**ava**S**cript **O**bject **N**otation) is a lightweight text format for representing data — objects, arrays, strings, numbers, booleans. It's the format almost every modern API uses to send/receive data, and it looks just like a JS object:

```
{
  "id": 1,
  "name": "Ahmed",
  "isStudent": true,
  "hobbies": ["reading", "coding"]
}
```

In your code, `JSON.stringify(obj)` turns a JS object into a JSON string (to send it), and `JSON.parse(str)` (or `response.json()`, as you'll see below) turns a JSON string back into a usable JS object.

XML (**Ex**tensible **M**arkup **L**anguage) is an older data format that uses opening/closing tags instead, similar to HTML:

```
<user>
  <id>1</id>
  <name>Ahmed</name>
  <isStudent>true</isStudent>
</user>
```

	JSON	XML
Looks like	JS objects	HTML-style tags
Size	Smaller, less "noise"	More verbose (opening + closing tags)
Used today	The default for almost all modern APIs	Still used in some older/enterprise systems (e.g. SOAP APIs)

> *You'll only work with **JSON** in this course — XML is here just so you recognize it if you ever see it.*

---

3. HTTP Methods

When your app talks to an API, it does so over **HTTP**, and every request specifies a **method** describing *what kind of action* it wants to perform:

Method	Purpose	Example
`GET`	**Read** data (doesn't change anything on the server)	Get a list of posts
`POST`	**Create** new data	Add a new post
`PUT`	**Replace** an existing item entirely	Overwrite a whole post
`PATCH`	**Update** part of an existing item	Change just a post's title
`DELETE`	**Remove** data	Delete a post

You'll see `GET` and `POST` used the most as a beginner — `PUT` / `PATCH` / `DELETE` follow the exact same `fetch` pattern, just with a different `method` value.

---

4. fetch

`fetch` is a built-in function used to make requests to an API.

```
fetch('https://jsonplaceholder.typicode.com/users')
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error(error));
```

Using `async/await` (cleaner, recommended):

```
const getUsers = async () => {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/us
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
};
```

---

5. GET and POST Requests (in Code)

Here's `GET` and `POST` (from the HTTP Methods table above) actually written with `fetch`:

GET — retrieve data

```
const getPosts = async () => {
  const response = await fetch('https://jsonplaceholder.typicode.com/post
  const data = await response.json();
  return data;
};
```

POST — send data

```
const createPost = async () => {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      title: 'My First Post',
      body: 'This is the content',
    }),
  });
  const data = await response.json();
  console.log(data);
};
```

---

6. Loading and Error Handling

Always show the user what's happening — loading, success, or error.

```tsx
import { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator } from 'react-native';

function UserList() {
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [users, setUsers] = useState<any[]>([]);

  useEffect(() => {
    const fetchUsers = async () => {
      try {
        const response = await fetch('https://jsonplaceholder.typicode.co
        if (!response.ok) throw new Error('Failed to fetch');
        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError('Something went wrong');
      } finally {
        setLoading(false);
      }
    };

    fetchUsers();
  }, []);

  if (loading) return <ActivityIndicator size="large" />;
  if (error) return <Text>{error}</Text>;

  return (
    <View>
      {users.map((user) => (
        <Text key={user.id}>{user.name}</Text>
      ))}
    </View>
```

```
  );
}
```

---

7. Typing API Responses

Define an interface that matches the shape of the data you expect from the API.

```
interface User {
  id: number;
  name: string;
  email: string;
}

const [users, setUsers] = useState<User[]>([]);

const fetchUsers = async () => {
  const response = await fetch('https://jsonplaceholder.typicode.com/users
  const data: User[] = await response.json();
  setUsers(data);
};
```

> *This gives you autocomplete (e.g. `user.name`) and catches typos/mistakes early.*

---

8. AsyncStorage

`AsyncStorage` lets you save small amounts of data **on the device** so it stays there even after closing the app (like a mini local database).

Install

```
npx expo install @react-native-async-storage/async-storage
```

9. Saving and Retrieving Data

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Save data (must be a string, so objects need JSON.stringify)
const saveName = async (name: string) => {
  try {
    await AsyncStorage.setItem('username', name);
  } catch (error) {
    console.error('Error saving data', error);
  }
};

// Retrieve data
const getName = async () => {
  try {
    const name = await AsyncStorage.getItem('username');
    if (name !== null) {
      console.log('Saved name:', name);
    }
  } catch (error) {
    console.error('Error reading data', error);
  }
};

// Remove data
const removeName = async () => {
  await AsyncStorage.removeItem('username');
};
```

Saving Objects/Arrays

```
interface Task {
  id: string;
  title: string;
}

const saveTasks = async (tasks: Task[]) => {
  await AsyncStorage.setItem('tasks', JSON.stringify(tasks));
};

const loadTasks = async (): Promise<Task[]> => {
  const jsonValue = await AsyncStorage.getItem('tasks');
  return jsonValue ? JSON.parse(jsonValue) : [];
};
```

Quick Recap

- API = a way for your app to talk to a server
- **JSON** = the text format almost all APIs use to send data; **XML** = an older, tag-based alternative
- HTTP methods: `GET` (read), `POST` (create), `PUT` / `PATCH` (update), `DELETE` (remove)
- `fetch` = make GET/POST (and other) requests
- Always handle **loading** and **error** states
- Type API responses with interfaces
- `AsyncStorage` = save data on the device (survives app restarts)
- Objects/arrays must be converted with `JSON.stringify` / `JSON.parse`

Practice Task

1. Fetch a list of posts from `https://jsonplaceholder.typicode.com/posts` and display the titles in a `FlatList`.

2. Save the user's name using `AsyncStorage` on a "Welcome Screen" and show it again the next time the app opens.

Chapter 8 — Final Project: Task App

> © ***Eng. Abdallah Mahmoud*** *— React Native & TypeScript Course Notes*

This chapter combines everything from Chapters 1–7 into one complete beginner project: a **Task App** where users can search, add, edit (in place), delete, and complete tasks, with clean card styling, icons, and data saved locally.

Layout we're building:

- A `SafeAreaView` wrapping the whole screen, so nothing hides behind the notch or home indicator
- A **header** at the top with an icon "logo" and the title "Task App"
- A **search bar** below the header to filter the task list
- A scrollable **list of task cards**, each with a checkbox-style icon, the task text (tap to edit), and a trash icon
- An **input + Add/Update button pinned to the bottom** of the screen, shared by both "adding" and "editing"

---

0. TypeScript Interface for a Task

Start by defining the shape of a task.

```ts
// types.ts
export interface Task {
  id: string;
  title: string;
  completed: boolean;
}
```

---

1. Using Icons (`@expo/vector-icons`)

Every Expo project comes with `@expo/vector-icons` already installed — no extra install step needed. It bundles several popular icon sets (Ionicons, MaterialIcons, FontAwesome, and more) as ready-to-use components.

```tsx
import { Ionicons } from '@expo/vector-icons';

<Ionicons name="trash-outline" size={22} color="red" />
<Ionicons name="checkmark-circle" size={24} color="green" />
<Ionicons name="clipboard-outline" size={28} color="#4F46E5" />
```

Each icon is just a component: `name` picks the icon from that set, `size` and `color` style it like any other prop. You can browse all available icon names on the Expo icon directory.

We'll use `Ionicons` for:

- A **clipboard icon** as the app's logo in the header
- A **checkmark-circle** (filled when done, outline when not) to mark a task complete
- A **trash icon** to delete a task
- A **search icon** inside the search bar

---

2. Wrapping the Screen in SafeAreaView

Before laying out the header, wrap the **whole screen** in a `SafeAreaView` (Chapter 4, Section 2) instead of a plain `View`, so the header never sits under the status bar/notch, and the bottom input bar never sits under the home indicator.

```
import { SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  return (
    <SafeAreaView style={styles.screen} edges={['top', 'bottom']}>
      {/* Header, search bar, list, and bottom bar all go inside here */}
    </SafeAreaView>
  );
}
```

Because the outer `SafeAreaView` already handles the top/bottom insets, the header and bottom bar **no longer need a manual `paddingTop: 55` hack** — that was only ever a rough guess at the status bar's height.

---

3. Header with Logo

Instead of a separate image file, we use an icon as a simple "logo" next to the title.

```
import { View, Text, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';

function Header() {
  return (
    <View style={styles.header}>
      <Ionicons name="clipboard-outline" size={26} color="#4F46E5" />
      <Text style={styles.headerTitle}>Task App</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
    paddingVertical: 16,
    paddingHorizontal: 20,
    backgroundColor: '#fff',
    borderBottomWidth: 1,
    borderBottomColor: '#eee',
  },
  headerTitle: { fontSize: 22, fontWeight: 'bold', color: '#111' },
});
```

> *If you'd rather use a real picture instead of an icon, swap the `<Ionicons>` for `<Image source={require('./assets/logo.png')} style={{ width: 28, height: 28 }} />` (see Chapter 4, Section 4).*

---

4. Search Bar (Filtering the List)

A search input at the top filters which tasks are shown, without touching the actual `tasks` array.

```
import { useState } from 'react';
import { View, TextInput, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';

const [search, setSearch] = useState('');

function SearchBar({ value, onChangeText }: { value: string; onChangeText
  return (
    <View style={styles.searchBar}>
      <Ionicons name="search" size={18} color="#999" />
      <TextInput
        style={styles.searchInput}
        placeholder="Search tasks..."
        value={value}
        onChangeText={onChangeText}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  searchBar: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#f1f1f4',
    borderRadius: 10,
    paddingHorizontal: 12,
    marginHorizontal: 20,
    marginTop: 14,
    gap: 8,
  },
  searchInput: { flex: 1, paddingVertical: 10, fontSize: 15 },
});
```

Filtering the list is just a plain array `.filter()` before passing data to `FlatList`:

```
const filteredTasks = tasks.filter((task) =>
  task.title.toLowerCase().includes(search.toLowerCase())
);
```

---

5. Add Tasks (Bottom Input Bar)

The input and button live in a bar **pinned to the bottom** of the screen (`position: 'absolute', bottom: 0`), so it's always reachable regardless of list length.

> ***Note:*** *the bar no longer needs extra bottom padding to clear the phone's home indicator — the outer* `SafeAreaView` *from Section 2 already handles that for us.*

```
const addTask = () => {
  if (input.trim() === '') return; // basic validation

  const newTask: Task = {
    id: Date.now().toString(),
    title: input,
    completed: false,
  };

  setTasks([...tasks, newTask]);
  setInput('');
};
```

```
<View style={styles.bottomBar}>
  <TextInput
    style={styles.bottomInput}
    placeholder="Add a new task..."
    value={input}
    onChangeText={setInput}
  />
  <TouchableOpacity style={styles.addButton} onPress={addTask}>
    <Ionicons name="add" size={24} color="#fff" />
  </TouchableOpacity>
</View>
```

```
const styles = StyleSheet.create({
  bottomBar: {
    position: 'absolute',
    bottom: 0,
    left: 0,
    right: 0,
    flexDirection: 'row',
    alignItems: 'center',
    padding: 14,
    backgroundColor: '#fff',
    borderTopWidth: 1,
    borderTopColor: '#eee',
    gap: 10,
  },
  bottomInput: {
    flex: 1,
    backgroundColor: '#f1f1f4',
    borderRadius: 10,
    paddingHorizontal: 14,
    paddingVertical: 10,
    fontSize: 15,
  },
  addButton: {
    backgroundColor: '#4F46E5',
    width: 44,
    height: 44,
    borderRadius: 22,
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

6. Edit Tasks In Place (Same Input, Same Button)

Instead of a separate “Edit” row, tapping a task’s text **reuses the same bottom input**: it loads the task’s current title into `input`, remembers which task is being edited in `editingId`, and the button switches from “Add” to “Update”.

```
const [editingId, setEditingId] = useState<string | null>(null);

const startEditing = (task: Task) => {
  setEditingId(task.id);
  setInput(task.title); // load existing text into the same input
};

const saveTask = () => {
  if (input.trim() === '') return;

  if (editingId) {
    // We're editing an existing task
    setTasks(
      tasks.map((task) =>
        task.id === editingId ? { ...task, title: input } : task
      )
    );
    setEditingId(null);
  } else {
    // We're adding a brand-new task
    setTasks([...tasks, { id: Date.now().toString(), title: input, comple
  }

  setInput('');
};
```

The bottom bar’s button now calls `saveTask` and its icon/label depends on `editingId`:

```
<TouchableOpacity style={styles.addButton} onPress={saveTask}>
  <Ionicons name={editingId ? 'checkmark' : 'add'} size={24} color="#fff"
</TouchableOpacity>
```

And tapping a task's title (instead of a separate "Edit" button) starts editing:

```
<TouchableOpacity onPress={() => startEditing(item)} style={{ flex: 1 }}>
  <Text style={styles.taskText}>{item.title}</Text>
</TouchableOpacity>
```

> ***Why one shared input instead of two?*** *It matches how most real task apps behave (tap a task, its text drops into the composer, edit, and save) and avoids duplicating the* `TextInput` *UI for "add" vs "edit" — one state variable (* `editingId` *) is all that separates the two behaviors.*

---

7. Delete Tasks (Trash Icon)

```
const deleteTask = (id: string) => {
  setTasks(tasks.filter((task) => task.id !== id));
};
```

A trash icon, not a text button, sits at the end of each task card:

```
<TouchableOpacity onPress={() => deleteTask(item.id)}>
  <Ionicons name="trash-outline" size={20} color="#E11D48" />
</TouchableOpacity>
```

---

8. Mark Tasks as Completed (Icon + Strikethrough)

A tappable circle icon toggles `completed`; the strikethrough on the text stays exactly as before.

```
const toggleComplete = (id: string) => {
  setTasks(
    tasks.map((task) =>
      task.id === id ? { ...task, completed: !task.completed } : task
    )
  );
};
```

```
<TouchableOpacity onPress={() => toggleComplete(item.id)}>
  <Ionicons
    name={item.completed ? 'checkmark-circle' : 'ellipse-outline'}
    size={22}
    color={item.completed ? '#22C55E' : '#999'}
  />
</TouchableOpacity>

<Text
  style={{
    textDecorationLine: item.completed ? 'line-through' : 'none',
    color: item.completed ? 'gray' : '#111',
  }}
>
  {item.title}
</Text>
```

---

9. Clean Card Styling for Each Task

Rather than plain rows separated by a bottom border, each task is a **card**: white background, rounded corners, a light shadow, sitting on a soft gray screen background. This makes each task visually separate and easy to scan.

```
renderItem={({ item }) => (
  <View style={styles.taskCard}>
    <TouchableOpacity onPress={() => toggleComplete(item.id)}>
      <Ionicons
        name={item.completed ? 'checkmark-circle' : 'ellipse-outline'}
        size={22}
        color={item.completed ? '#22C55E' : '#999'}
      />
    </TouchableOpacity>

    <TouchableOpacity onPress={() => startEditing(item)} style={{ flex: 1
      <Text
        style={[styles.taskText, item.completed && styles.taskTextDone]}
      >
        {item.title}
      </Text>
    </TouchableOpacity>

    <TouchableOpacity onPress={() => deleteTask(item.id)}>
      <Ionicons name="trash-outline" size={20} color="#E11D48" />
    </TouchableOpacity>
  </View>
)}
```

```
const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#F5F6FA' }, // soft gray backgroun
  taskCard: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 14,
    marginHorizontal: 16,
    marginTop: 10,
    // subtle shadow (iOS)
    shadowColor: '#000',
    shadowOpacity: 0.06,
    shadowRadius: 6,
    shadowOffset: { width: 0, height: 2 },
    // subtle elevation (Android)
    elevation: 2,
  },
  taskText: { fontSize: 15, color: '#111' },
  taskTextDone: { textDecorationLine: 'line-through', color: '#9CA3AF' },
});
```

---

10. TypeScript Interfaces (Recap)

Using the `Task` interface everywhere keeps the whole app type-safe:

```
export interface Task {
  id: string;
  title: string;
  completed: boolean;
}

const [tasks, setTasks] = useState<Task[]>([]);
```

This means TypeScript will warn you if you ever try to use `task.titel` (typo) or forget the `completed` field.

---

11. Navigation (Optional Extension)

You can extend the app with a second screen, e.g. a “Task Details” screen.

```
type RootStackParamList = {
  TaskList: undefined;
  TaskDetails: { task: Task };
};

// From Task List
navigation.navigate('TaskDetails', { task: item });

// In Task Details Screen
function TaskDetailsScreen({ route }: any) {
  const { task } = route.params;
  return <Text>{task.title}</Text>;
}
```

---

12. Local Storage (Persisting Tasks)

Use `AsyncStorage` so tasks are saved even after closing the app.

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import { useEffect } from 'react';

// Save tasks whenever they change
useEffect(() => {
  AsyncStorage.setItem('tasks', JSON.stringify(tasks));
}, [tasks]);

// Load tasks when app starts
useEffect(() => {
  const loadTasks = async () => {
    const saved = await AsyncStorage.getItem('tasks');
    if (saved) setTasks(JSON.parse(saved));
  };
  loadTasks();
}, []);
```

Full Working Example

```
import { useState, useEffect } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  FlatList,
  StyleSheet,
  KeyboardAvoidingView,
  Platform,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import AsyncStorage from '@react-native-async-storage/async-storage';

interface Task {
  id: string;
  title: string;
  completed: boolean;
}

export default function App() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [input, setInput] = useState('');
  const [search, setSearch] = useState('');
  const [editingId, setEditingId] = useState<string | null>(null);

  // Load saved tasks on start
  useEffect(() => {
    AsyncStorage.getItem('tasks').then((saved) => {
      if (saved) setTasks(JSON.parse(saved));
    });
  }, []);
```

```
// Save tasks whenever they change
useEffect(() => {
  AsyncStorage.setItem('tasks', JSON.stringify(tasks));
}, [tasks]);

const saveTask = () => {
  if (!input.trim()) return;

  if (editingId) {
    setTasks(tasks.map((t) => (t.id === editingId ? { ...t, title: inpu
    setEditingId(null);
  } else {
    setTasks([...tasks, { id: Date.now().toString(), title: input, comp
  }
  setInput('');
};

const startEditing = (task: Task) => {
  setEditingId(task.id);
  setInput(task.title);
};

const toggleComplete = (id: string) => {
  setTasks(tasks.map((t) => (t.id === id ? { ...t, completed: !t.comple
};

const deleteTask = (id: string) => {
  setTasks(tasks.filter((t) => t.id !== id));
  if (editingId === id) {
    setEditingId(null);
    setInput('');
  }
};

const filteredTasks = tasks.filter((t) =>
  t.title.toLowerCase().includes(search.toLowerCase())
```

```
  );

  return (
    <SafeAreaView style={styles.screen} edges={['top', 'bottom']}>
      <KeyboardAvoidingView
        style={styles.screen}
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
      >
      {/* Header */}
      <View style={styles.header}>
        <Ionicons name="clipboard-outline" size={26} color="#4F46E5" />
        <Text style={styles.headerTitle}>Task App</Text>
      </View>

      {/* Search bar */}
      <View style={styles.searchBar}>
        <Ionicons name="search" size={18} color="#999" />
        <TextInput
          style={styles.searchInput}
          placeholder="Search tasks..."
          value={search}
          onChangeText={setSearch}
        />
      </View>

      {/* Task list */}
      <FlatList
        data={filteredTasks}
        keyExtractor={(item) => item.id}
        contentContainerStyle={{ paddingBottom: 100 }}
        renderItem={({ item }) => (
          <View style={styles.taskCard}>
            <TouchableOpacity onPress={() => toggleComplete(item.id)}>
              <Ionicons
                name={item.completed ? 'checkmark-circle' : 'ellipse-outl
                size={22}
```

```
              color={item.completed ? '#22C55E' : '#999'}
            />
          </TouchableOpacity>

          <TouchableOpacity
            onPress={() => startEditing(item)}
            style={{ flex: 1, marginHorizontal: 10 }}
          >
            <Text style={[styles.taskText, item.completed && styles.tas
              {item.title}
            </Text>
          </TouchableOpacity>

          <TouchableOpacity onPress={() => deleteTask(item.id)}>
            <Ionicons name="trash-outline" size={20} color="#E11D48" />
          </TouchableOpacity>
        </View>
      )}
    />

    {/* Bottom input bar (shared by Add and Edit) */}
    <View style={styles.bottomBar}>
      <TextInput
        style={styles.bottomInput}
        placeholder={editingId ? 'Update task...' : 'Add a new task...'
        value={input}
        onChangeText={setInput}
      />
      <TouchableOpacity style={styles.addButton} onPress={saveTask}>
        <Ionicons name={editingId ? 'checkmark' : 'add'} size={24} colo
      </TouchableOpacity>
    </View>
    </KeyboardAvoidingView>
  </SafeAreaView>
);
}
```

```
const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#F5F6FA' },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
    paddingVertical: 16,
    paddingHorizontal: 20,
    backgroundColor: '#fff',
    borderBottomWidth: 1,
    borderBottomColor: '#eee',
  },
  headerTitle: { fontSize: 22, fontWeight: 'bold', color: '#111' },
  searchBar: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#f1f1f4',
    borderRadius: 10,
    paddingHorizontal: 12,
    marginHorizontal: 20,
    marginTop: 14,
    gap: 8,
  },
  searchInput: { flex: 1, paddingVertical: 10, fontSize: 15 },
  taskCard: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 14,
    marginHorizontal: 16,
    marginTop: 10,
    shadowColor: '#000',
    shadowOpacity: 0.06,
    shadowRadius: 6,
```

```
    shadowOffset: { width: 0, height: 2 },
    elevation: 2,
  },
  taskText: { fontSize: 15, color: '#111' },
  taskTextDone: { textDecorationLine: 'line-through', color: '#9CA3AF' },
  bottomBar: {
    position: 'absolute',
    bottom: 0,
    left: 0,
    right: 0,
    flexDirection: 'row',
    alignItems: 'center',
    padding: 14,
    paddingBottom: 28,
    backgroundColor: '#fff',
    borderTopWidth: 1,
    borderTopColor: '#eee',
    gap: 10,
  },
  bottomInput: {
    flex: 1,
    backgroundColor: '#f1f1f4',
    borderRadius: 10,
    paddingHorizontal: 14,
    paddingVertical: 10,
    fontSize: 15,
  },
  addButton: {
    backgroundColor: '#4F46E5',
    width: 44,
    height: 44,
    borderRadius: 22,
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Quick Recap

This project used almost everything from the course:

- **State** (`useState`) to manage tasks, search text, input, and which task is being edited
- **Icons** (`@expo/vector-icons`) for the header logo, checkmarks, and the trash button — no image files needed
- **Tap-to-edit**: tapping a task's title reuses the same bottom input/button instead of a separate edit row
- **Search/filter**: a top search bar filters the list with a plain `.filter()` , without touching the underlying data
- **Card styling**: white rounded cards with a soft shadow on a light gray screen background
- **Events** to handle add/update/delete/complete
- **TypeScript interfaces** for type-safe tasks
- **FlatList** to render the task list
- **AsyncStorage** to save tasks locally
- **StyleSheet & Flexbox** for layout, including a bottom-pinned input bar

Final Challenge

Extend the Task App with:

1. A "Clear All" button
2. A counter showing "X of Y tasks completed"
3. Categories/tags for tasks (e.g. Work, Personal), shown as small colored labels on each card
4. A second screen (using React Navigation) to view task details
5. A "Cancel" icon next to the input while editing, to exit edit mode without saving

---

© **Eng. Abdallah Mahmoud** — All rights reserved.